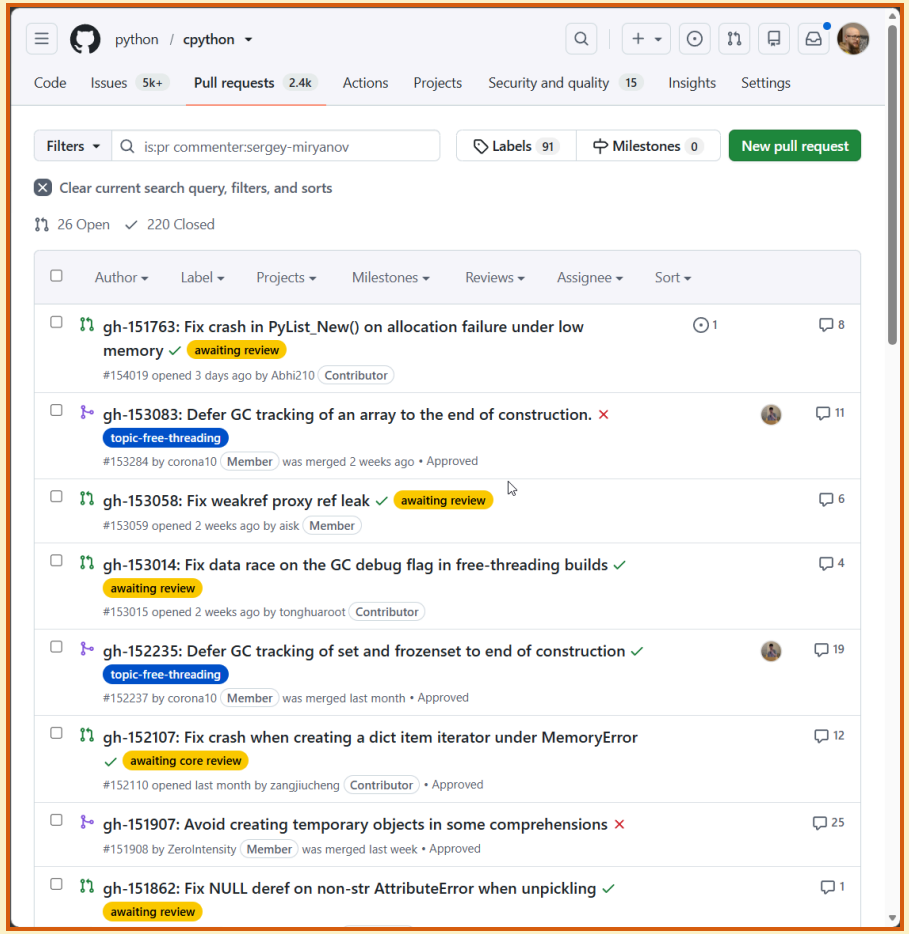
Monitoring Garbage Collector Events in Python

Yesterday, today, tomorrow?

Sergey Miryanov

PyCon Russia 2026

CPYTHON CONTRИBUTOR



Python Developer's Guide

This guide is a comprehensive resource for contributing to Python – for both new and experienced contributors. It is maintained by the same community that maintains Python. We welcome your contributions!

Start with the area that best matches what you want to do. If you still have questions after reviewing the material in this guide, then the Core Python Mentorship group is available to help guide new contributors through the process.

Documentation	Code	Triage
• Helping with documentation • Getting started • Style guide • reStructuredText primer • Translating • Helping with the Developer's Guide • Guidelines for using AI tools	• Setup and building • Where to get help • Lifecycle of a pull request • Running and writing tests • Fixing "easy" issues (and beyond) • Following Python's development • Git bootcamp and cheat sheet • Development cycle • Guidelines for using AI tools	• Issue tracker • Triaging an issue • Helping triage issues • Experts index • GitHub labels • GitHub issues for BPO users • Triage Team

We **recommend** that sections of this guide be read as needed. You can stop where you feel comfortable and begin contributing immediately without reading and understanding everything. If you do choose to skip around within the guide, be aware that some sections build on each other, so you may need to backtrack for missing concepts or terminology.

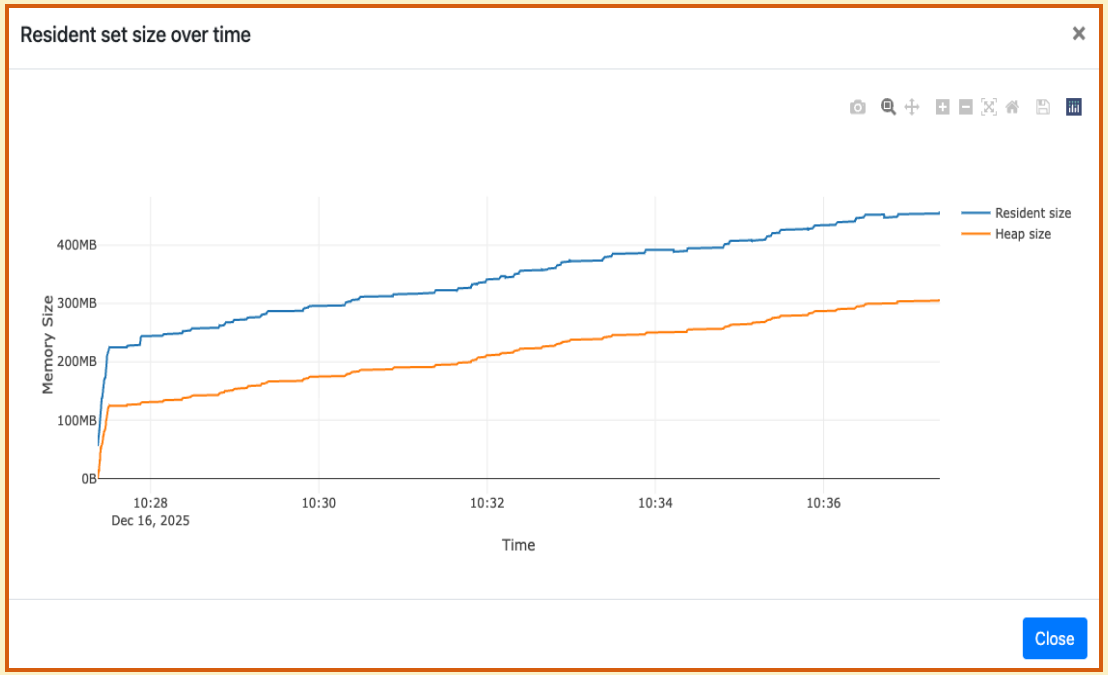
WHAT WE'LL TALK ABOUT

- Memory management
- Garbage collection (GC, Garbage Collector, Garbage Collection)
- Garbage collection events

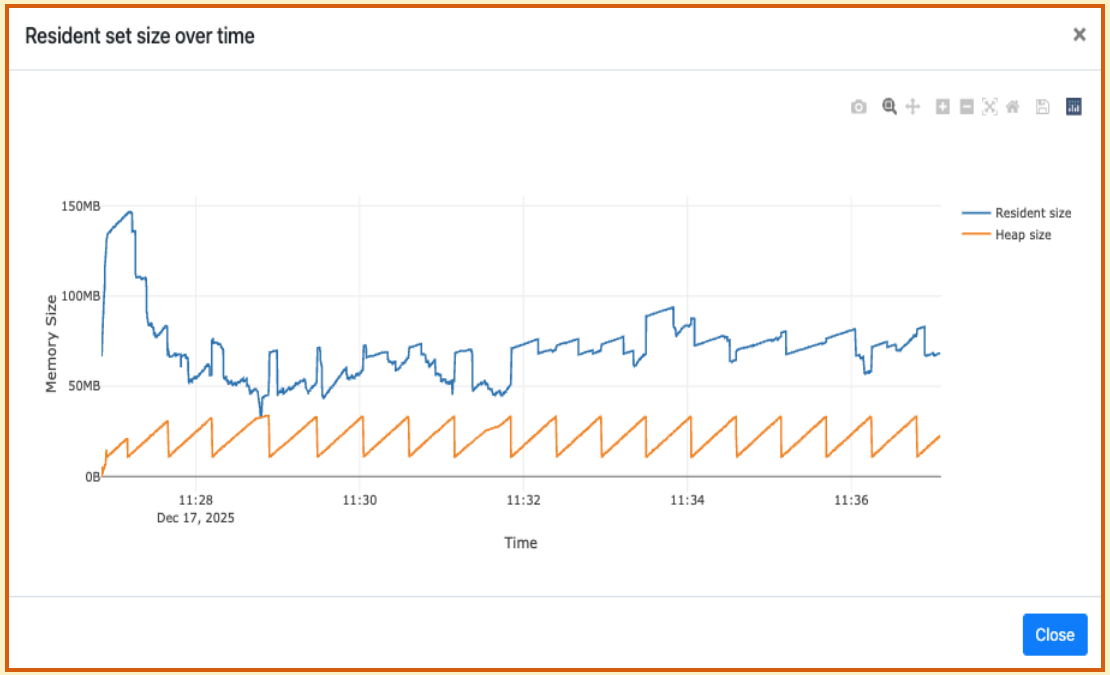
PROBLEM

https://github.com/python/cpython/issues/142516

3.14



3.13

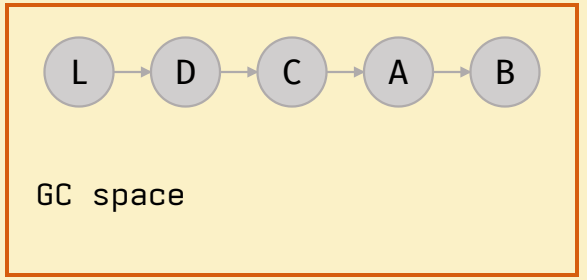


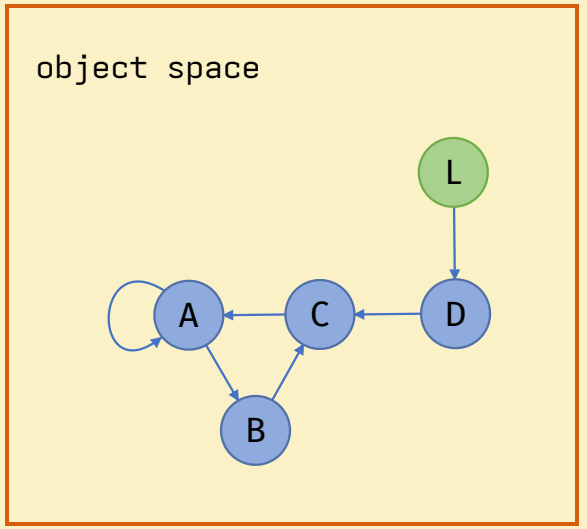
WHY DOES MEMORY USAGE GROW?

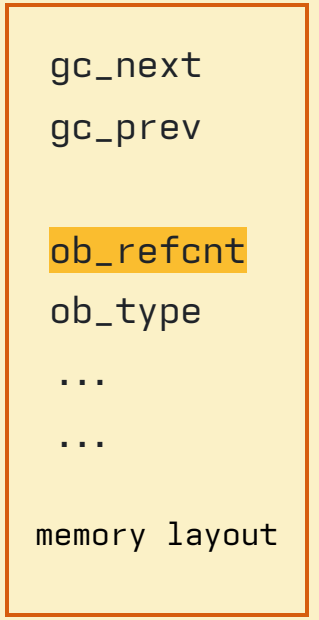
- A natural state?
- An internal leak?
- An external leak?
- A bug in the Python runtime/stdlib?
- A bug in the GC?

WHAT IS A MEMORY LEAK?

- Forgot to update the reference count
- Didn't free an unmanaged memory block
- Extended the object's lifetime
- The garbage collector didn't break cycles



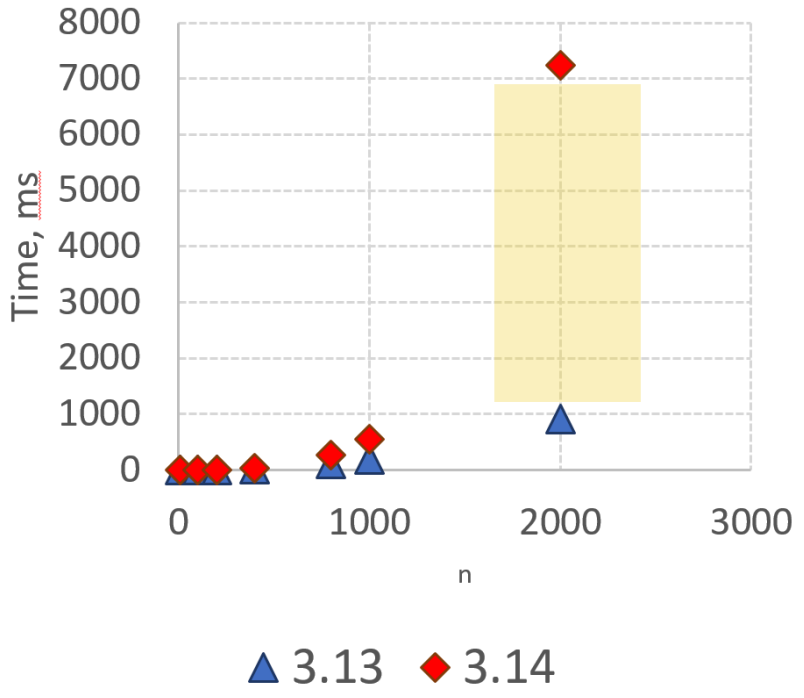




WHAT IS THE GC RESPONSIBLE FOR?

- Deduce unreachable objects
- Calls finalizers
- Handles weak references
- Breaks cycles
- *Deletes objects*

```
def test(n=2000):
    d = {}
    for i in range(n):
        for j in range(n):
            d[(i,j)] = i
```

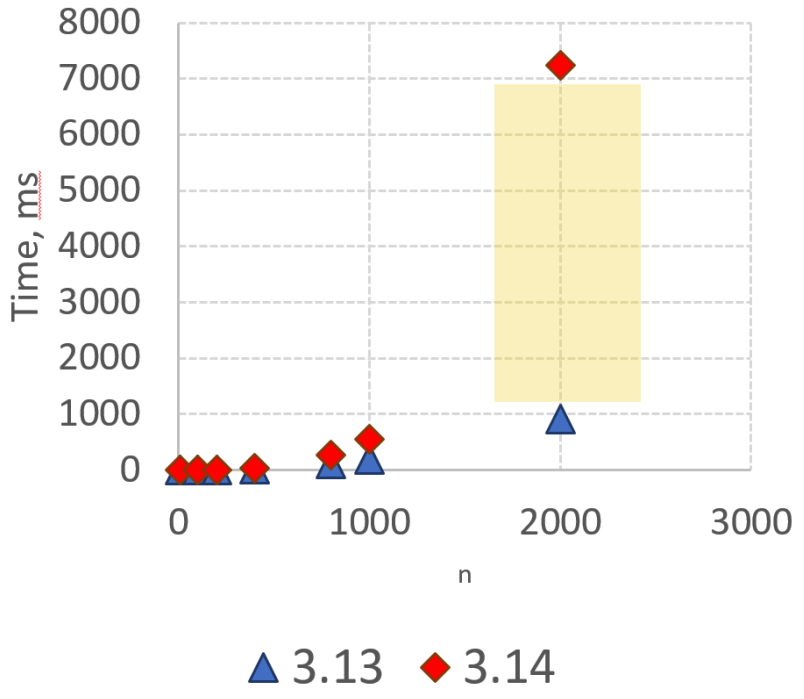


WHAT IS THE GC RESPONSIBLE FOR?

- **3.14.0-3.14.4**
  - **Identifies live objects**
  - **Fills the increment**
- Deduce unreachable objects
- Calls finalizers
- Handles weak references
- Breaks cycles
- *Deletes objects*

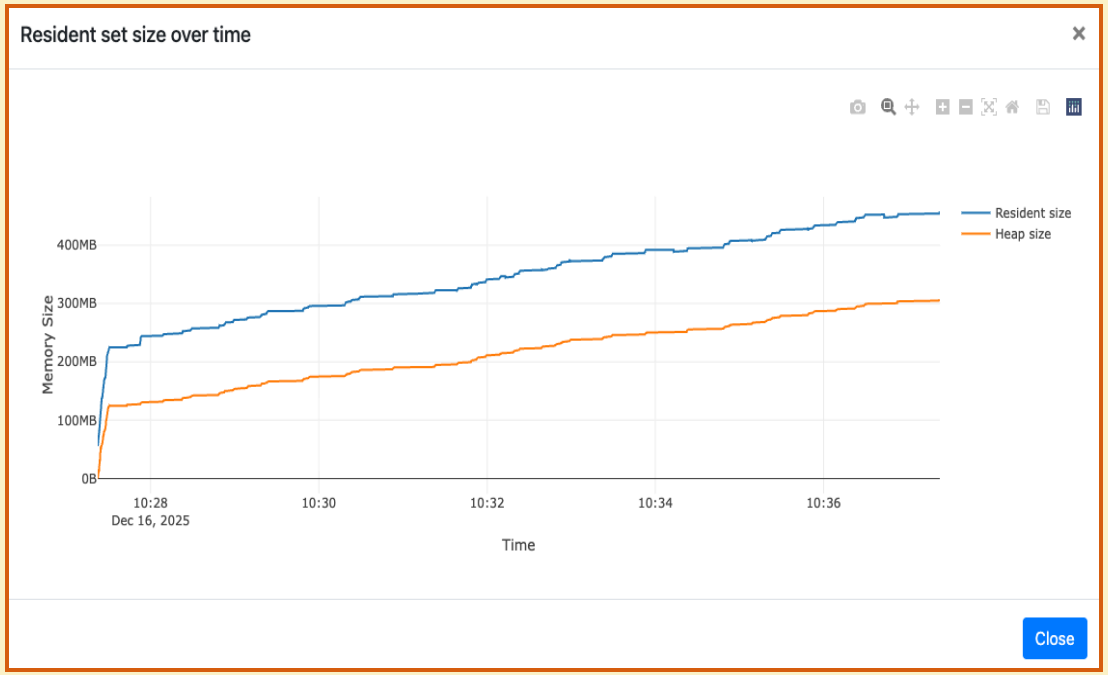
More details here →

```
def test(n=2000):
    d = {}
    for i in range(n):
        for j in range(n):
            d[(i,j)] = i
```



https://github.com/python/cpython/issues/142516

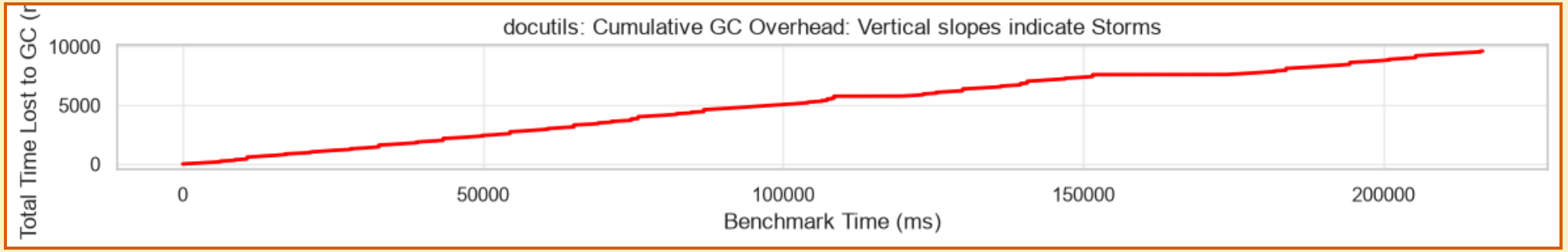
3.14




```
gc: collectable <Response 0x7fd8e3c4cc20>
  ⟶ <bytes 0x561a014e9218 rc=3221225472> <b''>
  ⟶ <IdentityDecoder 0x7fd8e3c4d010 rc=1> <<httpx._decoders.IdentityDecoder obje
  ⟶ <datetime.timedelta 0x7fd8e551a490 rc=1> <datetime.timedelta(microseconds=32
  ⟶ <int 0x561a014e71f8 rc=3221225472> <0>
  ⟶ <Request 0x7fd8e3cf4ec0 rc=1> <<Request('HEAD', 'http://www.google.com/')>>
  ⟶ <str 0x7fd8e52c9650 rc=17> <'utf-8'>
  ⟶ <dict 0x7fd8e3c2c540 rc=1> <{'http_version': b'HTTP/1.1', 'reason_phrase': b
  ⟶ <Headers 0x7fd8e3d6ae00 rc=1> <Headers([('content-type', 'text/html; charset
  ⟶ <list 0x7fd8e3c2c900 rc=1> <[]>
  ⟶ <bool 0x561a014a73c0 rc=3221225472> <True>
  ⟶ <bool 0x561a014a73c0 rc=3221225472> <True>
  ⟶ <NoneType 0x561a014be300 rc=3221225472> <None>
  ⟶ <int 0x561a014e8af8 rc=3221225472> <200>
  ⟶ <BoundAsyncStream 0x7fd8e3c4cec0 rc=1> <<httpx._client.BoundAsyncStream obje
    ⟶ <Response 0x7fd8e3c4cc20 rc=1> <<Response [200 OK]>>
      ⟶ <bytes 0x561a014e9218 rc=3221225472> <b''>
      ⟶ <IdentityDecoder 0x7fd8e3c4d010 rc=1> <<httpx._decoders.IdentityDecoder
      ⟶ <datetime.timedelta 0x7fd8e551a490 rc=1> <datetime.timedelta(microsecond
      ⟶ <int 0x561a014e71f8 rc=3221225472> <0>
      ⟶ <Request 0x7fd8e3cf4ec0 rc=1> <<Request('HEAD', 'http://www.google.com/'
      ⟶ <str 0x7fd8e52c9650 rc=17> <'utf-8'>
      ⟶ <dict 0x7fd8e3c2c540 rc=1> <{'http_version': b'HTTP/1.1', 'reason_phrase
      ⟶ <Headers 0x7fd8e3d6ae00 rc=1> <Headers([('content-type', 'text/html; cha
      ⟶ <list 0x7fd8e3c2c900 rc=1> <[]>
      ⟶ <bool 0x561a014a73c0 rc=3221225472> <True>
      ⟶ <bool 0x561a014a73c0 rc=3221225472> <True>
      ⟶ <NoneType 0x561a014be300 rc=3221225472> <None>
      ⟶ <int 0x561a014e8af8 rc=3221225472> <200>
      ⟶ <BoundAsyncStream 0x7fd8e3c4cec0 rc=1> <<httpx._client.BoundAsyncStream
      ⟶ <type 0x561a239058b0 rc=30> <<class 'httpx.Response'>>
    ⟶ <float 0x7fd8e3cdd3b0 rc=1> <39037.104077293>
    ⟶ <AsyncResponseStream 0x7fd8e3cf6120 rc=1> <<httpx._transports.default.Asyn
    ⟶ <type 0x561a23924160 rc=15> <<class 'httpx._client.BoundAsyncStream'>>
  ⟶ <type 0x561a239058b0 rc=30> <<class 'httpx.Response'>>
```

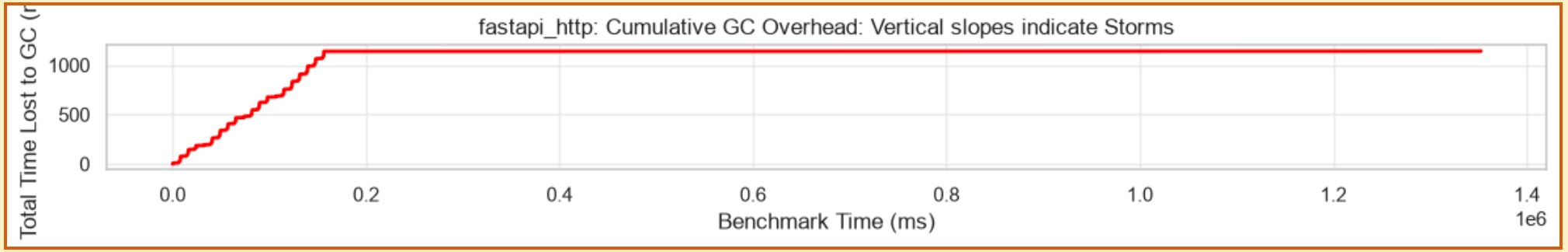
GC STORM

- Long/frequent pauses
- Increased memory/CPU usage
- Frequent full garbage collection



GC STORM

- Long/frequent pauses
- Increased memory/CPU usage
- Frequent full garbage collection



SUMMARY

- Causes of memory-usage growth
- What counts as a leak
- How reference counting works
- How the garbage collector works
- How the garbage collector affects the program
- What GC storm/thrashing is

TOOLS

- Problem
- **Tools**
- New API
- gcmon
- Perfetto
- Benchmarks
- Backports

TOOLS

- **In-process**
  - `tracemalloc`
  - `pympler`
  - APM (ddtrace, dynatrace, ...)
- **Out-of-process**
  - `memray`
  - `austin`, `tachyon`
  - `pyroscope`, `perforator`

- CPU
- Allocations
- Snapshots
- Cycles
- Lifetime
- Leaks
- Number of GC pauses
- Number of collected objects

THE GC MODULE

- **What's wrong with `gc.callbacks`**
  - Intrusive
  - C->Python interop
  - Subinterpreters
  - Heisenberg effect
- **What's wrong with `gc.get_stats`**
  - Intrusive
  - When
  - Subinterpreters

NEW API

- Problem
- Tools
- **New API**
- gcmon
- Perfetto
- Benchmarks
- Backports

GCMONITOR

- _remote_debugging.get_gc_stats
- _remote_debugging.GCMonitor
  - init(pid)
  - get_gc_stats

- External process
- Standard metrics
  - Collections
  - Collected
  - Uncollectable
  - *Candidates*
  - *Duration*
- *Number of live objects*
- Pause start and end time
- Subinterpreter support

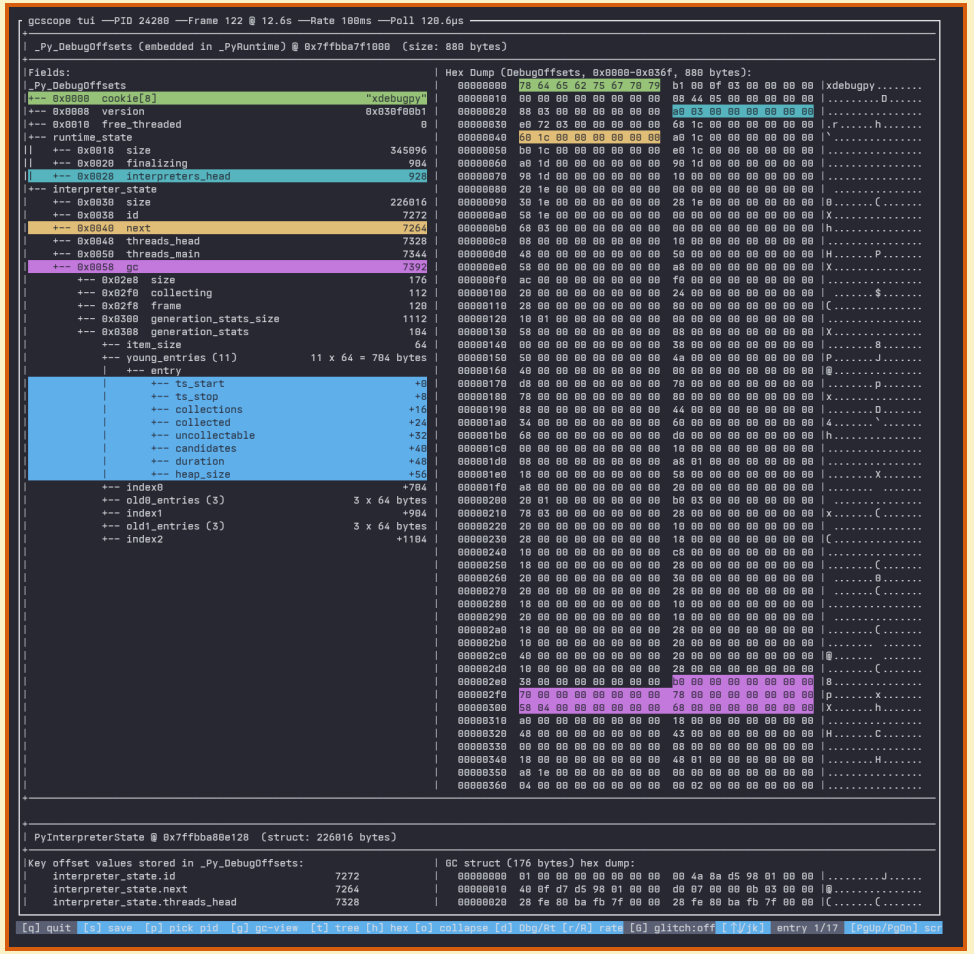
GCMONITOR

```
struct pyruntimestate {
    /* This field must be first to facilitate locating it by out of process
     * debuggers. Out of process debuggers will use the offsets contained in
     * this field to be able to locate other fields in several interpreter
     * structures in a way that doesn't require them to know the exact layout
     * of those structures.
     *
     * IMPORTANT:
     * This struct is **NOT** backwards compatible between minor version of the
     * interpreter and the members, order of members and size can change
     * between minor versions. This struct is only guaranteed to be stable
     * between patch versions for a given minor version of the interpreter.
     */
    _Py_DebugOffsets debug_offsets;
    // ...
    struct pyinterpreters {
        PyMutex mutex;
        /* The linked list of interpreters, newest first. */
        PyInterpreterState *head;
        PyInterpreterState *main;
        int64_t next_id;
    } interpreters;
    // ...
};
```

```
typedef struct _Py_DebugOffsets {
    char cookie[8] _Py_NONSTRING;
    uint64_t version;
    uint64_t free_threaded;
    struct _runtime_state {
        uint64_t size;
        uint64_t finalizing;
        uint64_t interpreters_head;
    } runtime_state;
    struct _interpreter_state {
        uint64_t size;
        uint64_t id;
        uint64_t next;
        uint64_t gc;
        // ...
    } interpreter_state;
    struct _gc {
        uint64_t size;
        uint64_t collecting;
        uint64_t frame;
        uint64_t generation_stats_size;
        uint64_t generation_stats;
    } gc;
} _Py_DebugOffsets;
```

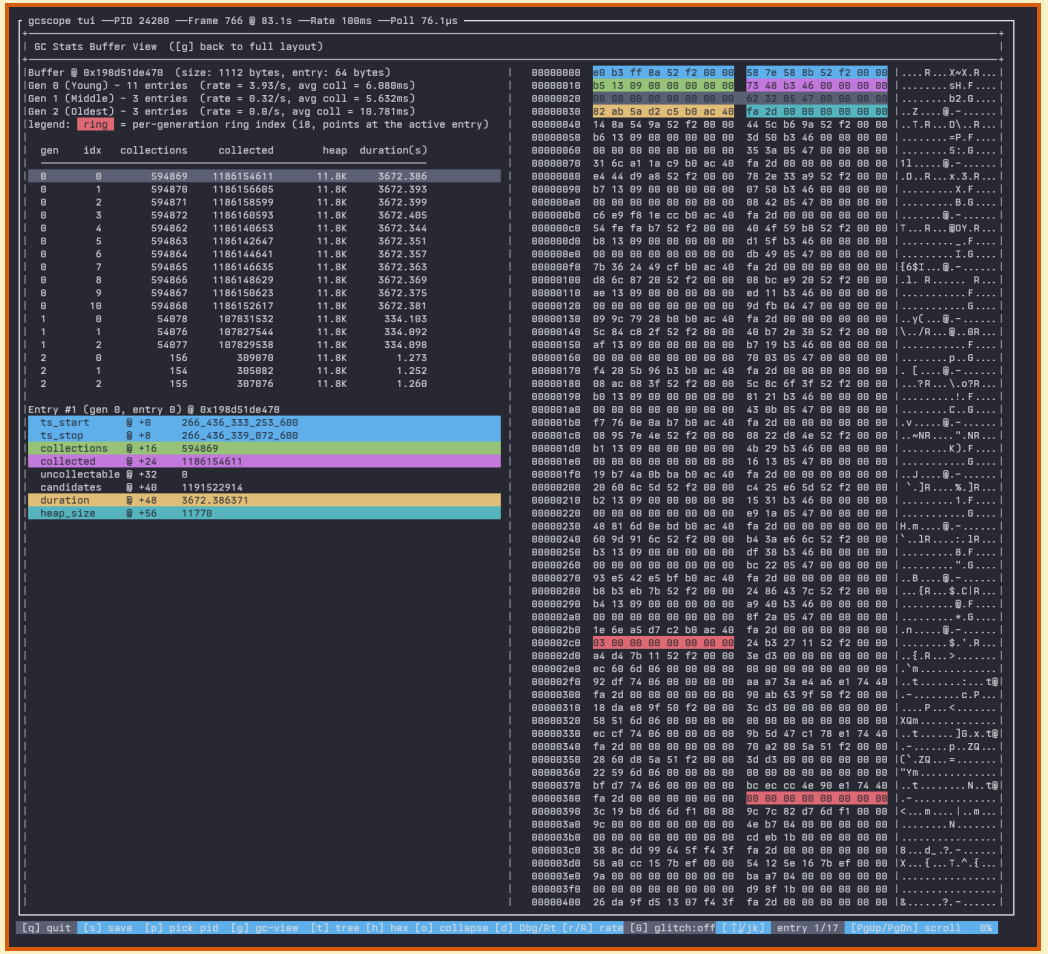
GCMONITOR

```
typedef struct _Py_DebugOffsets {
    char cookie[8] _Py_NONSTRING;
    uint64_t version;
    uint64_t free_threaded;
    struct _runtime_state {
        uint64_t size;
        uint64_t finalizing;
        uint64_t interpreters_head;
    } runtime_state;
    struct _interpreter_state {
        uint64_t size;
        uint64_t id;
        uint64_t next;
        uint64_t gc;
        // ...
    } interpreter_state;
    struct _gc {
        uint64_t size;
        uint64_t collecting;
        uint64_t frame;
        uint64_t generation_stats_size;
        uint64_t generation_stats;
    } gc;
} _Py_DebugOffsets;
```



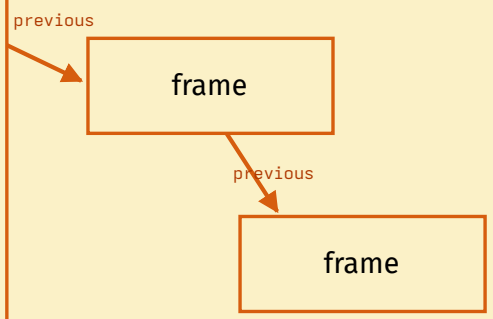
GC STATISTICS

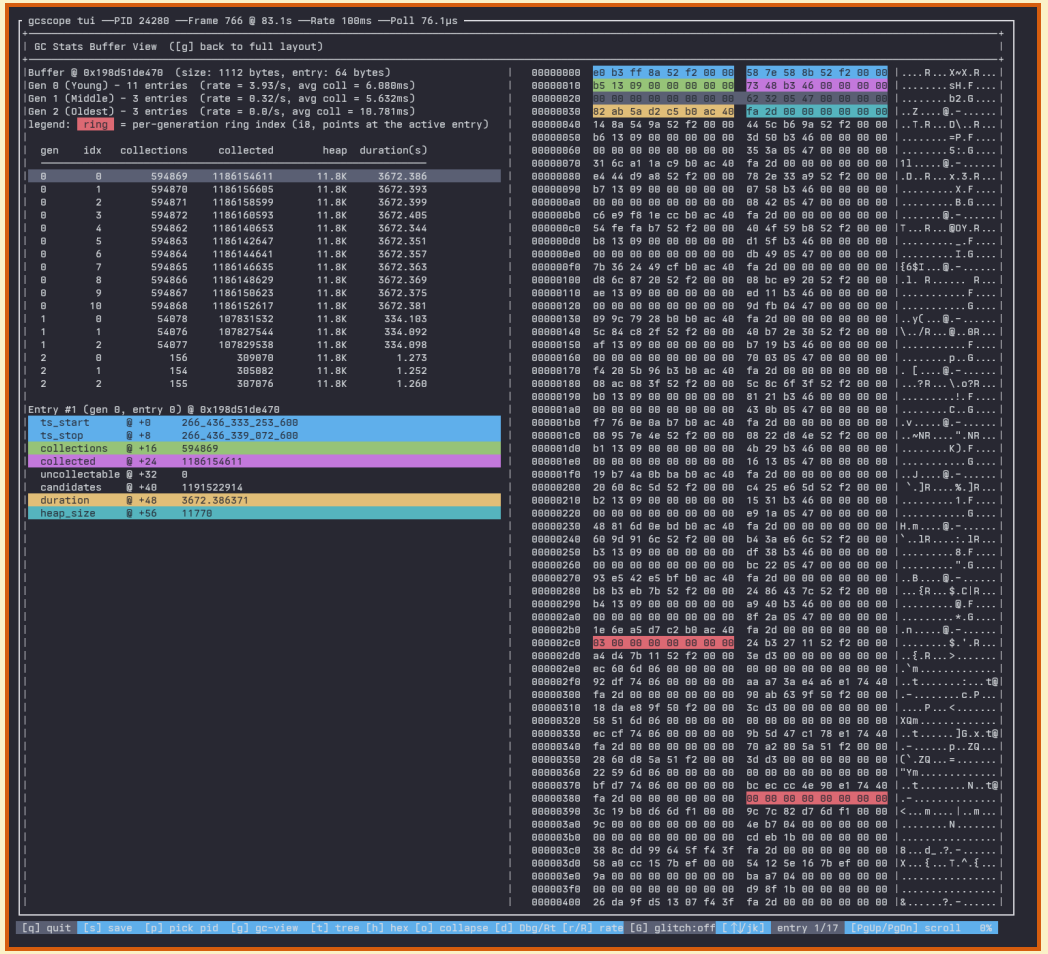
```
struct gc_generation_stats {
    PyTime_t ts_start;
    PyTime_t ts_stop;
    Py_ssize_t collections;
    Py_ssize_t collected;
    Py_ssize_t uncollectable;
    Py_ssize_t candidates;
    double duration;
    Py_ssize_t heap_size;
};
struct gc_young_stats_buffer {
    struct gc_generation_stats items[11];
    int8_t index;
};
struct gc_old_stats_buffer {
    struct gc_generation_stats items[3];
    int8_t index;
};
struct gc_stats {
    struct gc_young_stats_buffer young;
    struct gc_old_stats_buffer old[2];
};
```



PAUSES

```
struct _PyInterpreterFrame {
_PyStackRef f_executable;
struct _PyInterpreterFrame *previous;
_PyStackRef f_funcobj;
PyObject *f_globals;
PyObject *f_builtins;
PyObject *f_locals;
PyFrameObject *frame_obj;
_Py_CODEUNIT *instr_ptr;
_PyStackRef *stackpointer;
uint16_t return_offset;
char owner;
uint8_t visited;
/* Locals and stack */
_PyStackRef localsplus[1];
};
```





PITFALLS

- Non-Python processes
- Processes tree
- Race condition
  - Incomplete initialization
  - Partial read
- Elevated privileges
- GC storm/thrashing

SUMMARY

- Existing tools answer different questions
- Using the GC module distorts measurements
- The new API lets you look inside the GC without distortion
- The new API requires no pauses
- But there are pitfalls to keep in mind

GCMON

- Problem
- Tools
- New API
- **gcmon**
- Perfetto
- Benchmarks
- Backports

GCMON

- **Process management**
  - Child processes
  - Lifetime
  - Metadata
  - RSS
- **Extended metrics**
- **Export**
  - Various formats
- `pyperf` support
- Labels

- **Overhead**
  - Rate=100ms -> 1%
  - Rate=10ms -> 3-4%
- What about gcscope?

- Perfetto Binary Format
- Chrome Trace Format
- JSONL/stdout
- OpenTelemetry?

GCMON

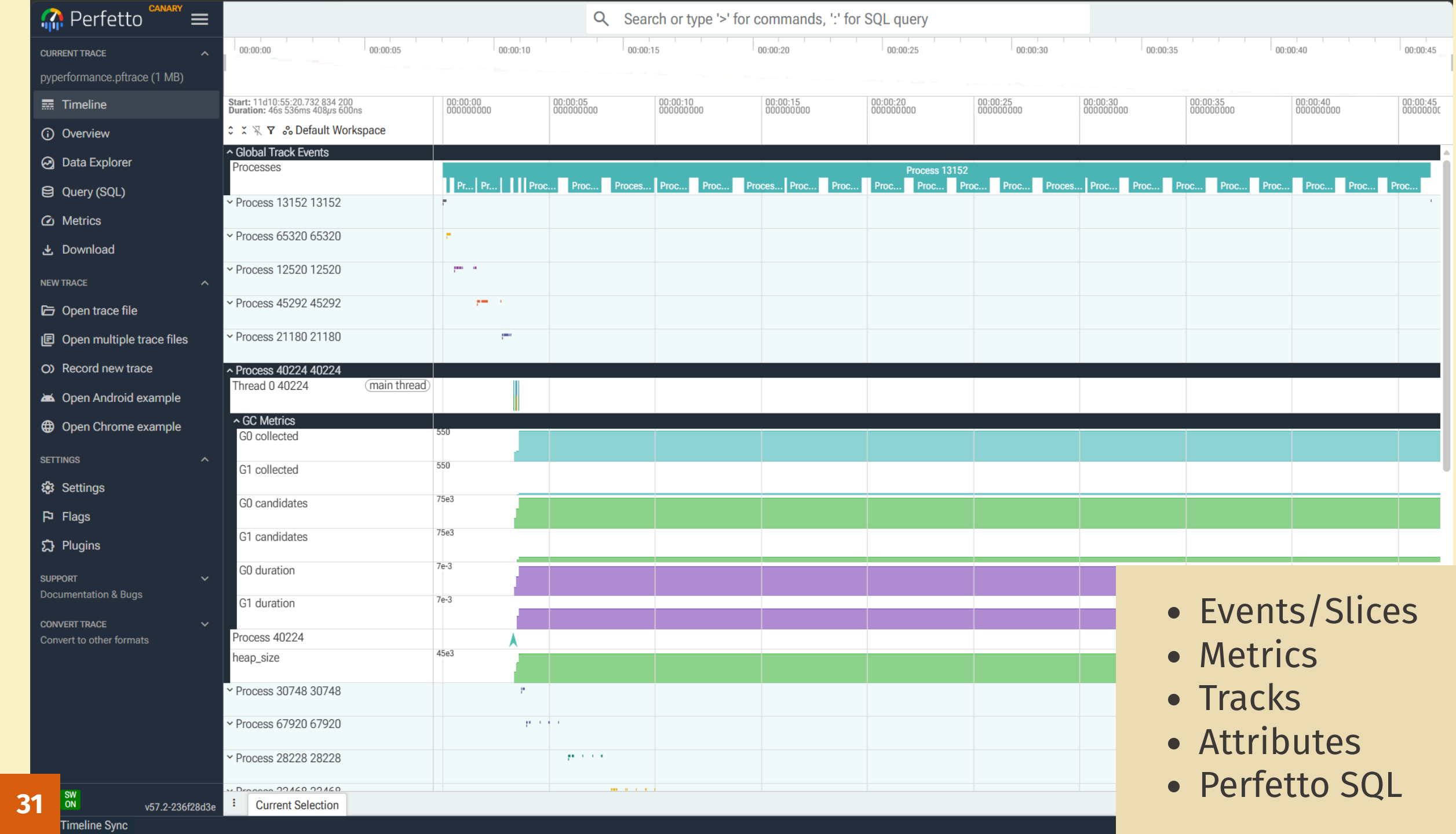
- **Standard metrics**
  - Collections
  - Collected
  - Uncollectable
  - *Candidates*
  - *Duration*
- *Number of live objects*
- Pause start and end time
- RSS

GCMON

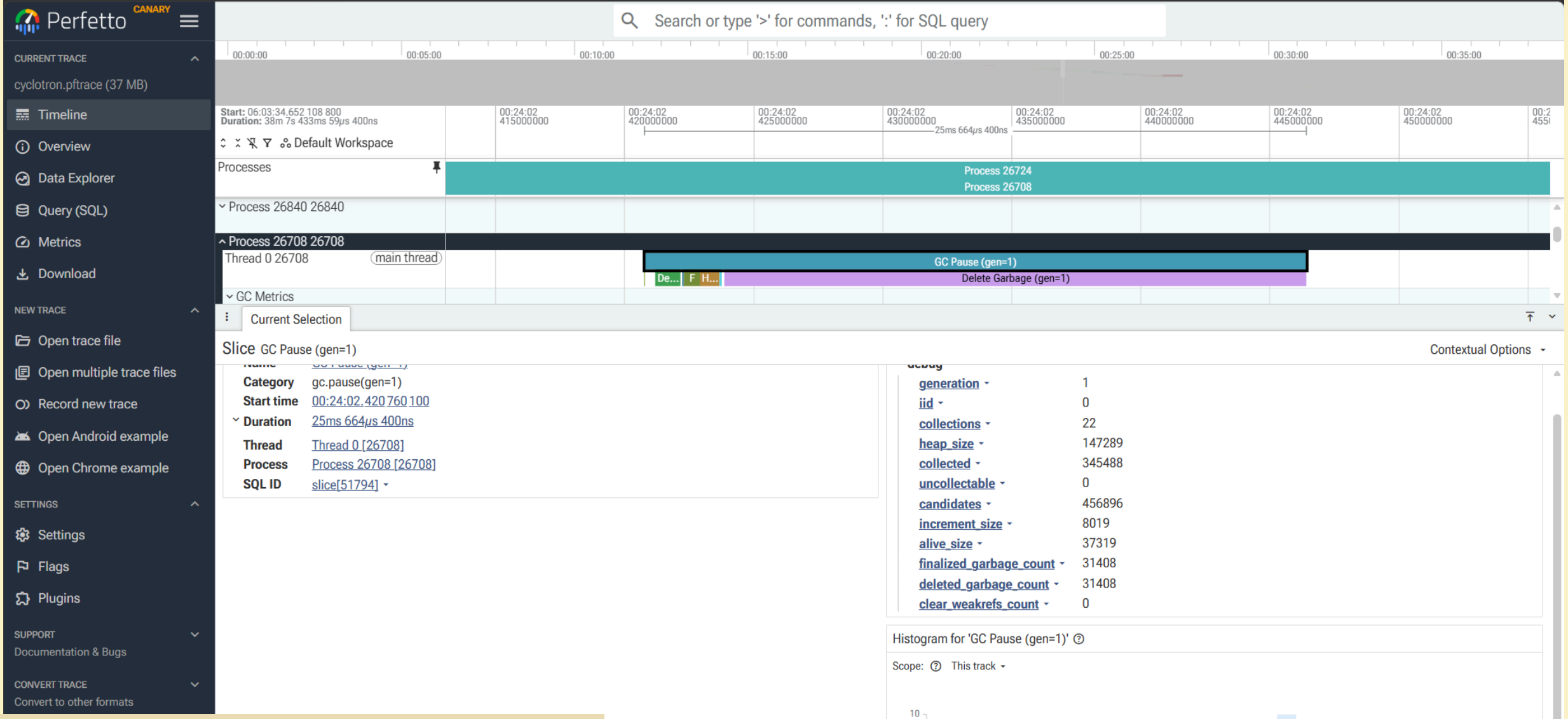
- **Debug data**
  - Mark Alive
  - Fill Increment
  - Deduce Unreachable
  - Handle Weakrefs Callbacks
  - Finalize Garbage
  - Handle Resurrected
  - Clear Weakrefs
  - Delete Garbage

PERFETTO

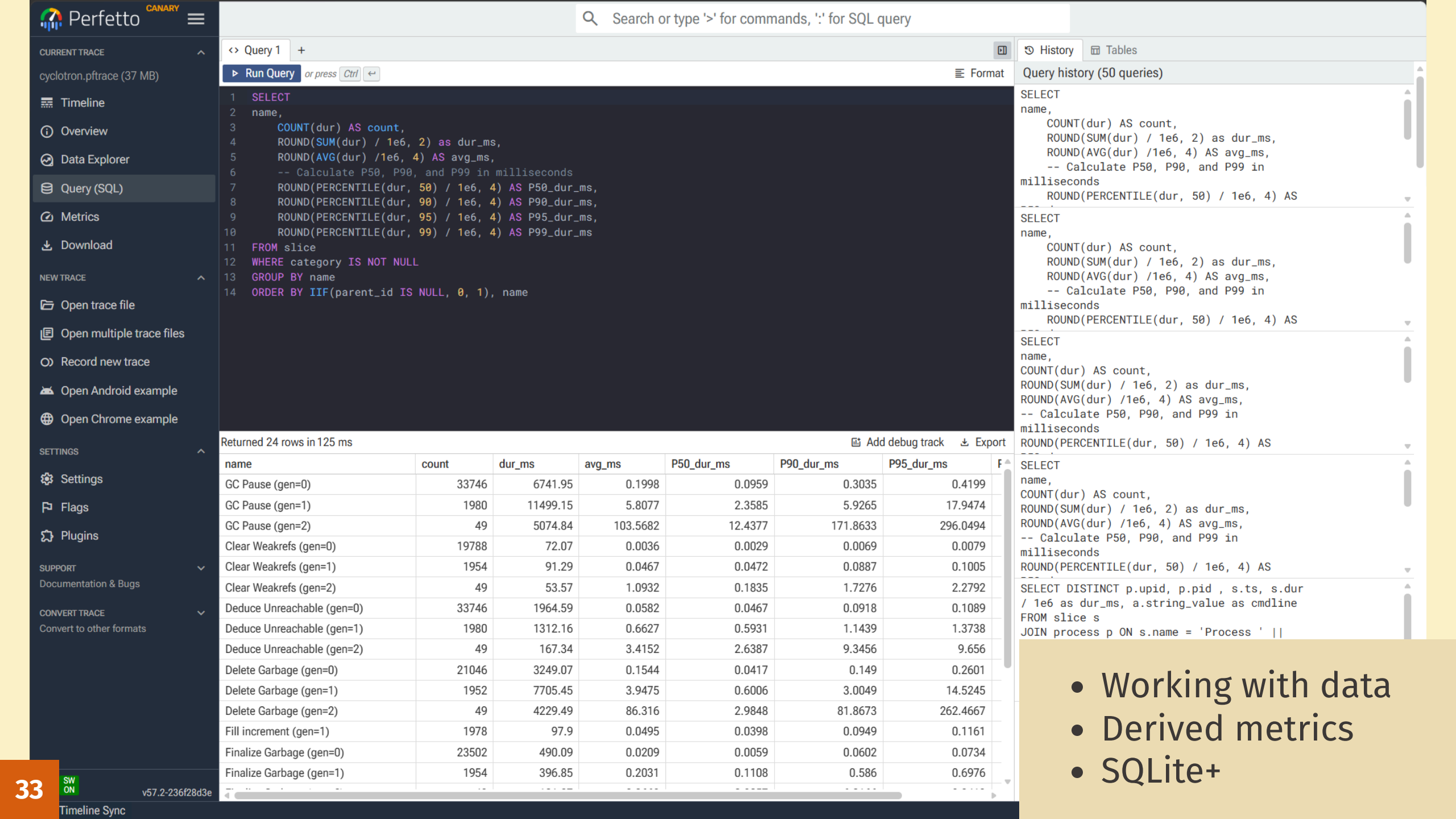
- Problem ●
- Tools ●
- New API ●
- gcmon ●
- **Perfetto** ●
- Benchmarks ○
- Backports ○



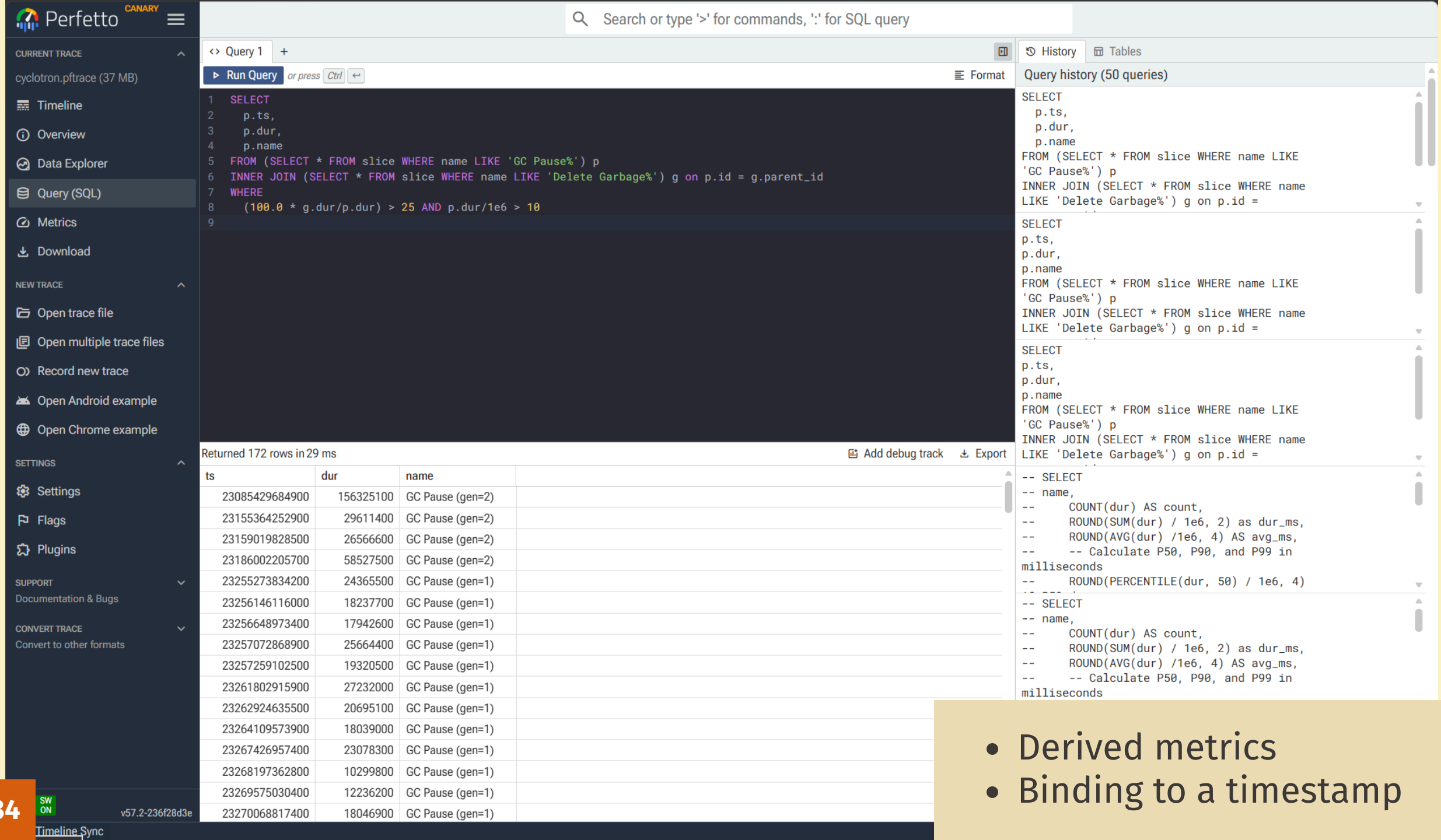
- Events/Slices
- Metrics
- Tracks
- Attributes
- Perfetto SQL



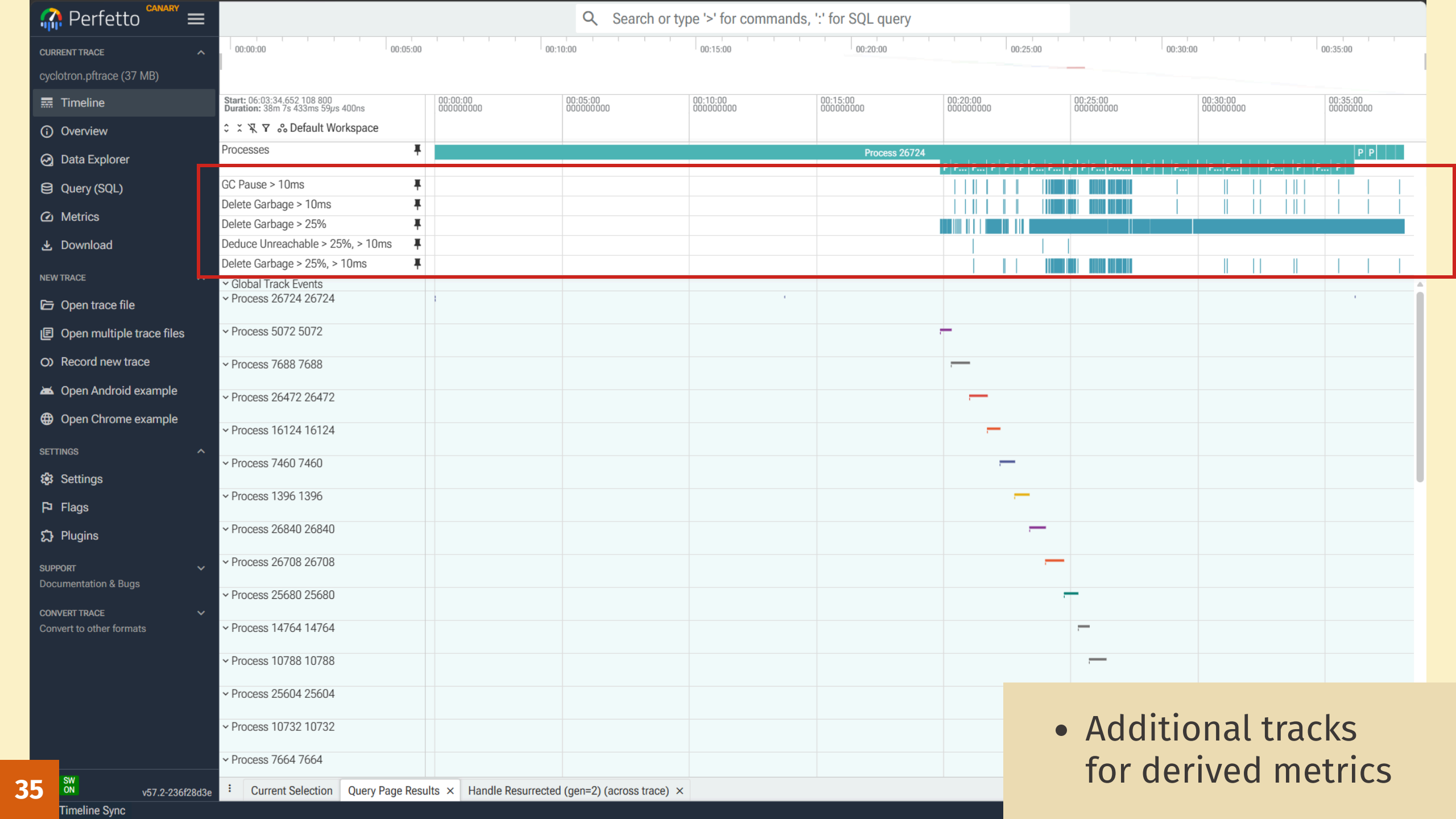
- Process minimap
- Attributes
- Garbage collection phases



- Working with data
- Derived metrics
- SQLite+



- Derived metrics
- Binding to a timestamp



- Additional tracks for derived metrics

SUMMARY

- `gcmon` — an example of using the new API
- Lets you obtain additional information
- Uses Perfetto to store and visualize the data
- Perfetto can be used for your own purposes

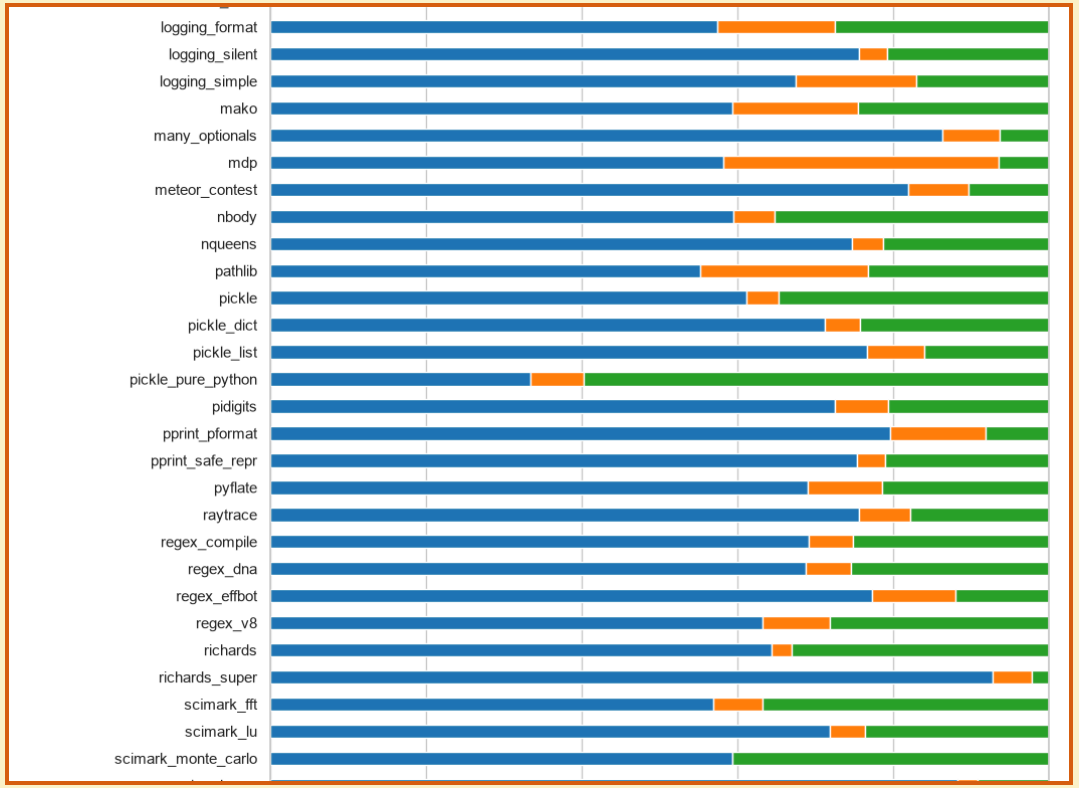
BENCHMARKS

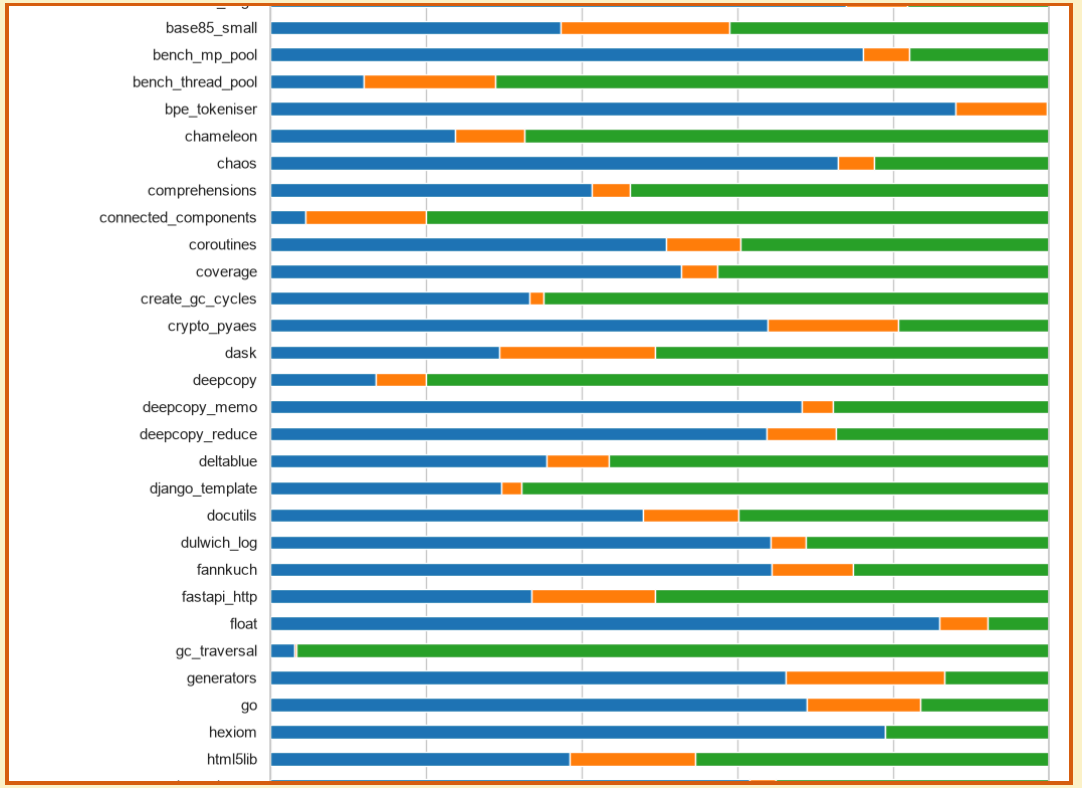
- Problem
- Tools
- New API
- gcmon
- Perfetto
- **Benchmarks**
- Backports

BENCHMARKS

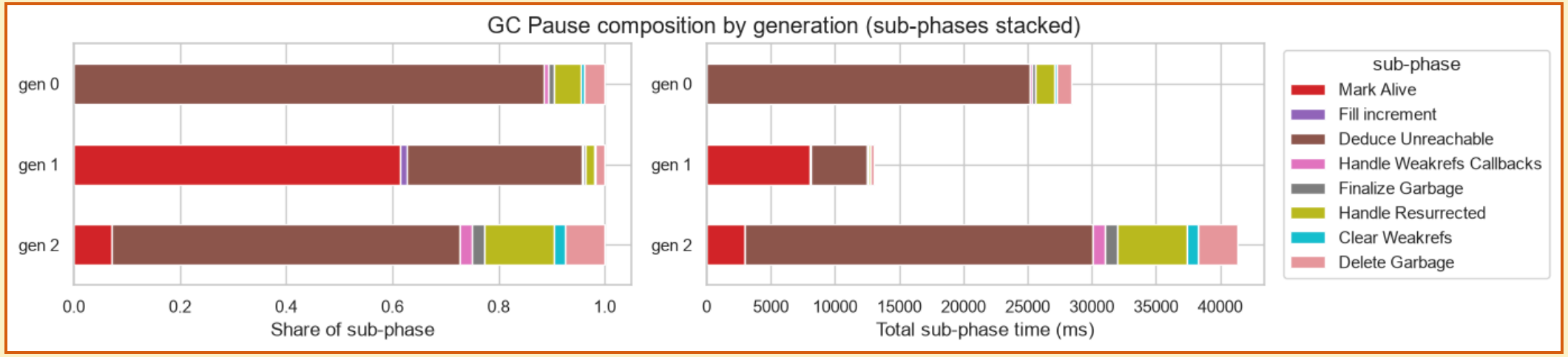
- Why benchmarks?
- Pyperformance
- Cyclotron
- Any other

WHICH GENERATION DOMINATES?

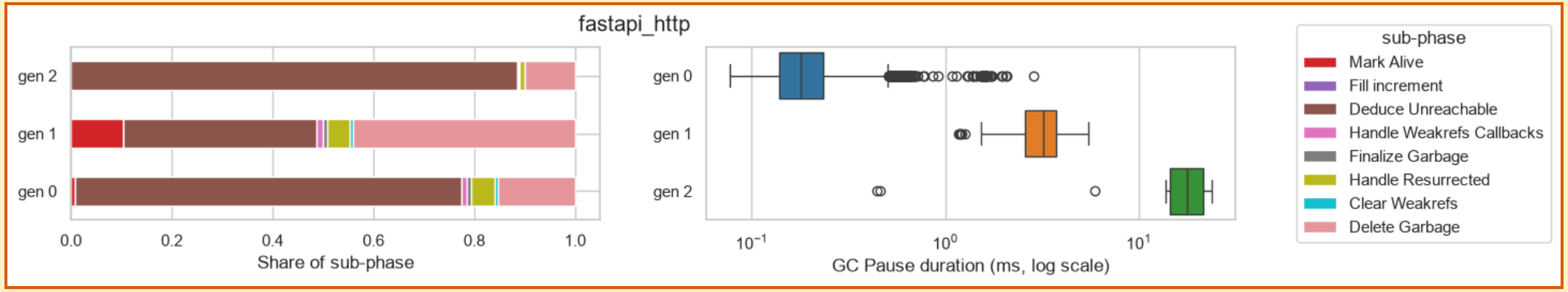




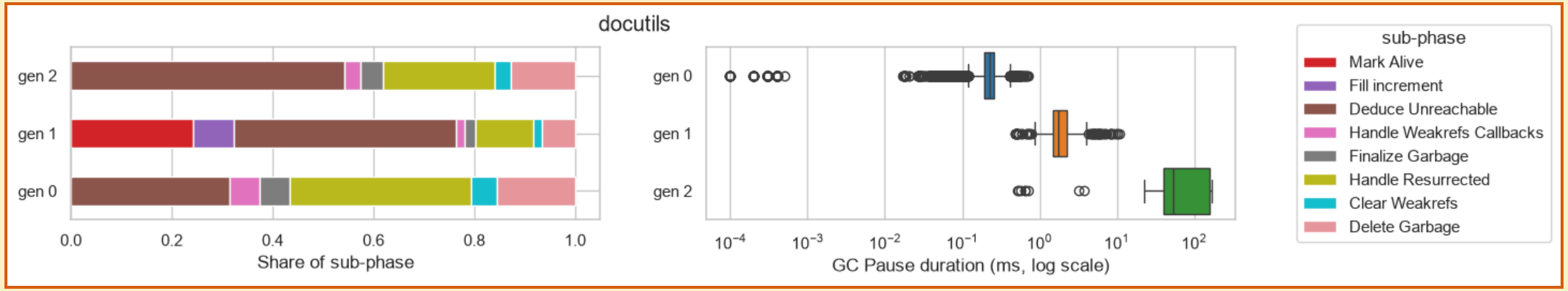
TYPICAL PHASE DISTRIBUTION



MANY OBJECTS WITH "HEAVY" DATA



FINALIZERS


docutils
gen 2
gen 1
gen 0
0.0
0.2
0.4
0.6
0.8
1.0
Share of sub-phase
gen 0
gen 1
gen 2
10^{-4}
10^{-3}
10^{-2}
10^{-1}
10^{0}
10^{1}
10^{2}
GC Pause duration (ms, log scale)
sub-phase
Mark Alive
Fill increment
Deduce Unreachable
Handle Weakrefs Callbacks
Finalize Garbage
Handle Resurrected
Clear Weakrefs
Delete Garbage

SUMMARY

- `gcmon` / the new API
  - can be used to profile production systems
  - can be used to analyze garbage collector behavior
- You can obtain information about possible tuning of production systems

BACKPORTS

- Problem
- Tools
- New API
- gcmon
- Perfetto
- Benchmarks
- **Backports**

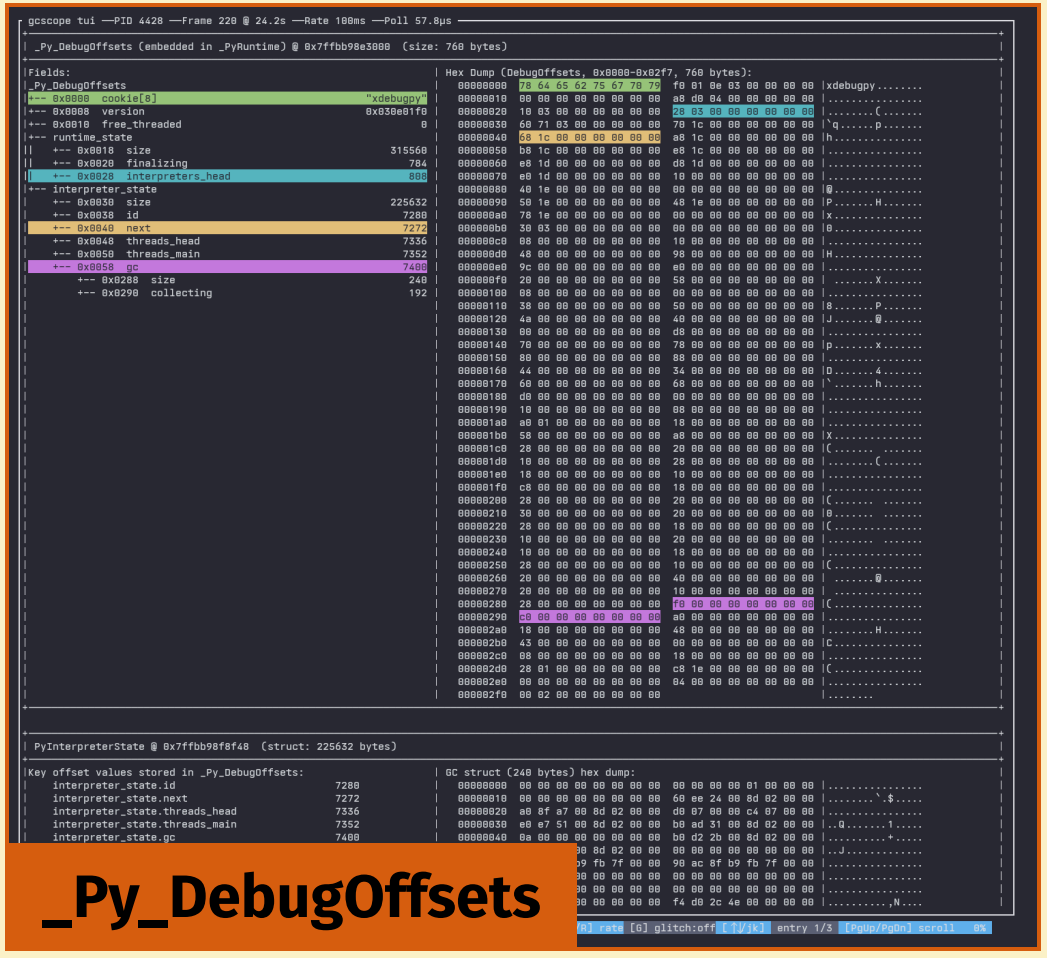
3.13-3.14

```
typedef struct _Py_DebugOffsets {
    char cookie[8] _Py_NONSTRING;
    uint64_t version;
    uint64_t free_threaded;
    struct _runtime_state {
        uint64_t size;
        uint64_t finalizing;
        uint64_t interpreters_head;
    } runtime_state;
    struct _interpreter_state {
        uint64_t size;
        uint64_t id;
        uint64_t next;
        uint64_t gc;
        // ...
    } interpreter_state;
    struct _gc {
        uint64_t size;
        uint64_t collecting;
    } gc;
} _Py_DebugOffsets;
```

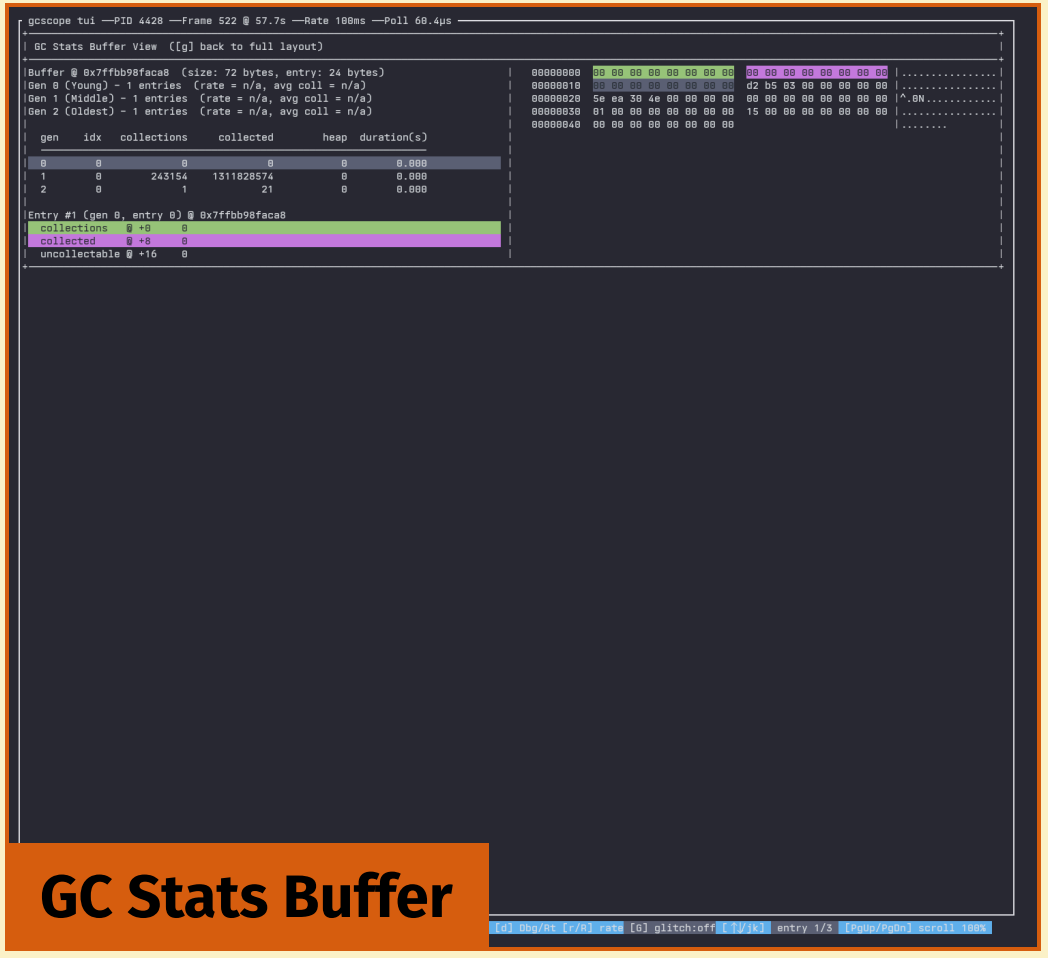
```
/* Running stats per generation */
struct gc_generation_stats {
    Py_ssize_t collections;
    Py_ssize_t collected;
    Py_ssize_t uncollectable;
};
```

```
struct _gc_runtime_state {
    PyObject *trash_delete_later;
    int trash_delete_nesting;
    /* Is automatic collection enabled? */
    int enabled;
    int debug;
    /* linked lists of container objects */
    struct gc_generation generations[NUM_GENERATIONS];
    PyGC_Head *generation0;
    /* a permanent generation which won't be collected */
    struct gc_generation permanent_generation;
    struct gc_generation_stats generation_stats[NUM_GENERATIONS];
    /* true if we are currently running the collector */
    int collecting;
    /* list of uncollectable objects */
    PyObject *garbage;
    /* a list of callbacks to be invoked when collection is performed */
    PyObject *callbacks;
};
```

3.13-3.14

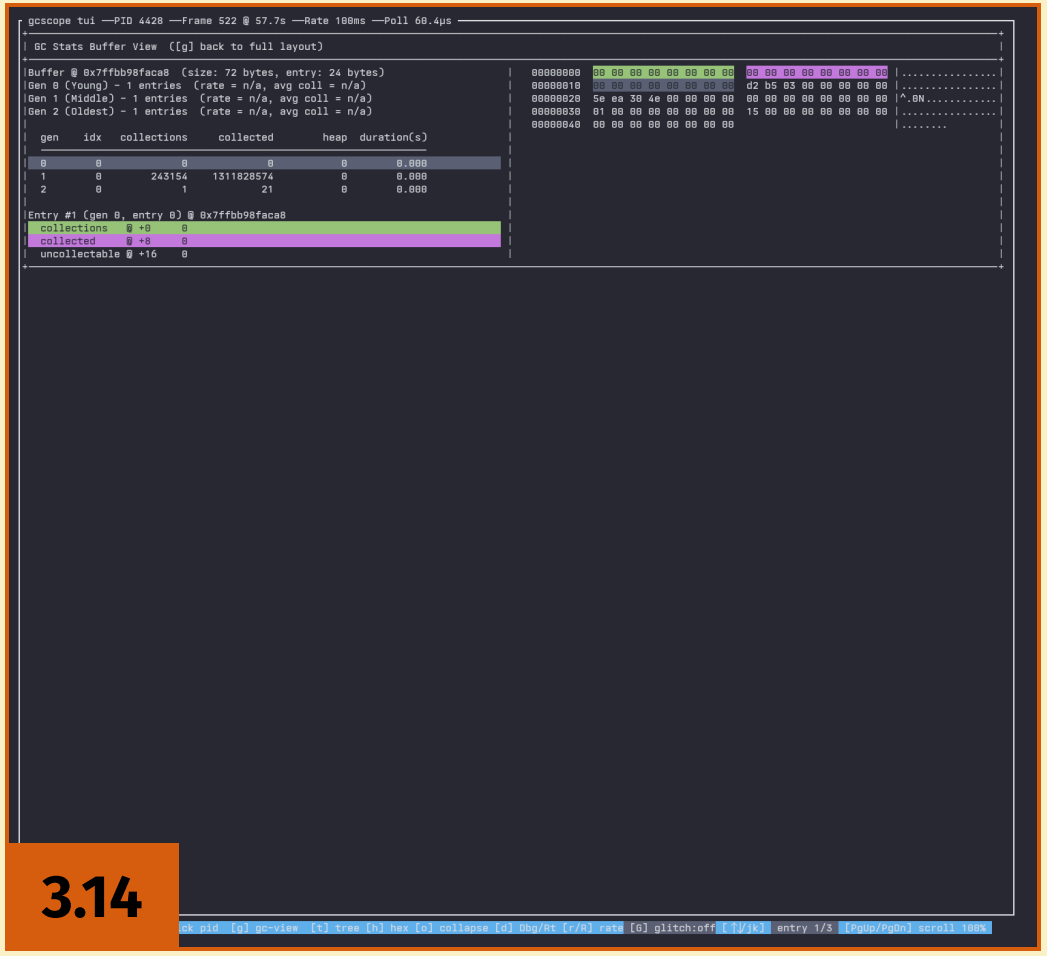


_Py_DebugOffsets



GC Stats Buffer

3.13-3.14



3.14

3.15+

3.8, 3.9-3.10, 3.11-3.12

- What if there's no marker, nothing at all — what do we do?
- How stable is it?
- What factors can affect it?
- How do we validate it?

3.8

CALL TO ACTION

- Which metrics are interesting and/or necessary for you?
- In what scenarios would you use these capabilities?
- Which formats and integrations interest you?
- What is your environment?
- What alerts do you envision?

Where to reach us:

- gcmon/issues
- discuss.python.org
- CPython/issues

THANK YOU FOR YOUR ATTENTION!

email: sergey.miryanov@gmail.com